# MyShelf

## Smart Inventory Management & Monitoring System

### Graduation Project Documentation — Part I

**Track:** DevOps Engineering
**Team size:** 4 members
**Status:** Draft for team review and alignment with official documentation guidelines

## Team Members and Proposed Responsibilities

*Role assignments are suggestions for planning, not confirmed final assignments. The team should approve or adjust them.*

| Team member | Proposed responsibility |
|---|---|
| Hazem El Sayed Abu El Wafa El Tayeb | System Architecture & Integration |
| Mohamed Gamal Fawzy Abdelzaher | Inventory Tracking Service |
| Bishoy Emad Fawzy Zakhary | Docker & Jenkins CI/CD |
| Mohamed Amer Farid Mohamed | Kubernetes & Infrastructure Automation |
| All Team Members | Testing, Integration & Documentation |

# 1. Project Planning & Management

## 1.1 Project Overview

MyShelf is a Smart Inventory Management & Monitoring System for tracking product quantities, identifying low-stock conditions, and providing reports through a web dashboard for managers. The system will be implemented as containerized services and delivered using DevOps practices. The supplied Project 3 brief explicitly requires an inventory tracking service, a low-stock alert service, a manager dashboard, automated CI/CD, Kubernetes deployment, Prometheus monitoring, Nginx dashboard access, Terraform infrastructure scripts for AWS EC2/S3/VPC, Ansible configuration automation, and a Git repository.

> ***Source alignment:*** *The scope and deliverables in this revision follow the Project 3 description supplied by the team: inventory tracking service, low-stock alert service, manager dashboard, Jenkins CI/CD, Kubernetes, Prometheus monitoring, Nginx, Terraform for AWS EC2/S3/VPC, Ansible, and Git repository. Detailed implementation choices and proposed targets are identified as proposals where the source brief does not specify them.*

## 1.2 Problem Statement

Businesses can experience inefficient inventory tracking, leading to stock-outs or overstocking. MyShelf aims to centralize inventory quantity tracking, surface low-stock conditions through alerts, and provide reports for managers. The graduation-project emphasis is on automation and DevOps practices; **AI is not required**.

## 1.3 Goals and Objectives

1. Track inventory quantities through a dedicated inventory tracking service.
2. Detect low-stock conditions and provide alerts through a separate alert service.
3. Provide a web dashboard for managers to inspect inventory status and reports.
4. Package application services as Docker containers.
5. Automate code integration and delivery/update steps with a Jenkins CI/CD pipeline.
6. Deploy the containerized services to Kubernetes.
7. Monitor system and container metrics using Prometheus, with Grafana used for visualization as listed in the supplied tool list.
8. Provide dashboard access through Nginx.
9. Use Terraform scripts to define/provision AWS EC2, S3, and VPC resources.
10. Use Ansible scripts to automate server and service configuration.
11. Maintain the project code and scripts in a shared Git repository.

## 1.4 Project Scope

### In scope — required by the supplied Project 3 brief

- Inventory tracking service (Dockerized).
- Low-stock alert service.
- Web dashboard for managers.
- Jenkins CI/CD pipeline for automated updates.
- Kubernetes deployment for scalable services.
- Prometheus monitoring for system and container metrics.
- Nginx for dashboard access.
- Terraform scripts for AWS EC2, S3, and VPC.
- Ansible scripts for server/service configuration.
- One Git repository containing the project code and scripts.
- Grafana is included in the supplied tool list; the proposed use is to visualize relevant monitoring data.

## Proposed implementation details — confirm with the instructor/team

- Product fields may include product ID/SKU, name, category, current quantity, low-stock threshold, and last-updated timestamp.
- The low-stock rule may be `quantity <= minimum_stock_threshold`.
- The dashboard may show product lists, low-stock items, and basic stock summary reports.
- A database and application framework are not specified in the supplied brief and must be selected by the team.
- Alert delivery method (dashboard indicator, in-app message, email, or another method) is not specified; begin with the simplest demonstrable alert behavior unless the rubric says otherwise.
- The exact Kubernetes cluster setup and AWS deployment topology are not specified in the supplied brief.
- No AI features are planned because the source explicitly says AI is not required.

## Out of scope unless separately approved

- AI/ML forecasting.
- Full ERP/accounting/purchasing system.
- Barcode/RFID hardware integration.
- Automatic purchase-order generation.
- SMS/email integrations if not required.
- Production-grade multi-region disaster recovery or compliance certification.

## Proposed Technical Architecture

The following is a high-level design proposal based on the required components; it is not a claim that the instructor has mandated a specific API, database, or cluster topology.

```
Manager / Browser
       |
     Nginx  ---> Web Dashboard
                    |       \
                    |        \--> Reports / Low-stock status
                    v
         Inventory Tracking Service
                    |
              Product data store
                    |
           Stock-change evaluation
                    v
             Low-Stock Alert Service

Delivery and operations layer:
Git -> Jenkins CI/CD -> Docker images -> Kubernetes workloads
Terraform -> AWS VPC / EC2 / S3 (approved scope)
Ansible -> server and service configuration
Prometheus -> system/container metrics -> Grafana visualization (proposed)
```

**Proposed request/data flow:** a manager views or updates inventory through the dashboard; the dashboard communicates with the inventory service through an agreed interface. The inventory service validates and persists the change. The alert service evaluates the updated quantity against the configured threshold and exposes an alert state for the dashboard. The exact synchronous/asynchronous communication pattern and database technology remain design decisions.

**Proposed delivery flow:** a change is pushed to Git; Jenkins checks out the revision, runs configured validation/tests, builds Docker images, and performs the agreed deployment/update steps. Kubernetes runs the application workloads. Prometheus scrapes configured metrics endpoints, while Grafana may be used to visualize the collected data. Nginx provides the intended dashboard entry point. Terraform and Ansible handle infrastructure definition and configuration automation respectively.

## 1.5 Required Deliverables and Evidence

| # | Deliverable from Project 3 brief | Proposed evidence for review/demo |
|---|---|---|
| 1 | Inventory tracking service (Dockerized) | Service source code, Dockerfile, and successful container run |
| 2 | Alert service for low stock | Alert service code and tests showing low-stock behavior |
| 3 | Web dashboard for managers | Dashboard showing inventory, low-stock items, and reports |
| 4 | CI/CD pipeline for automated updates | Jenkinsfile/pipeline configuration and successful pipeline run |
| 5 | Kubernetes deployment for scalable services | Kubernetes manifests/configuration and deployed workloads |
| 6 | Prometheus monitoring for system and container metrics | Prometheus configuration/targets and collected metrics |
| 7 | Nginx for dashboard access | Nginx configuration and working dashboard access path |
| 8 | Terraform scripts for AWS EC2, S3, VPC | Terraform files and plan/apply evidence for approved resources |
| 9 | Ansible scripts for server/service configuration | Playbook(s) and evidence of a successful configuration run |
| 10 | Git repository for code and scripts | Shared GitHub repository with README and organized folders |

The evidence column is a proposed way to demonstrate completion; it does not replace the official grading rubric.

## 1.6 Proposed Timeline

This is an estimate for team planning, not an official deadline. Adjust it to the instructor's schedule and the actual team capacity.

| Phase | Activities | Estimate | Exit criterion |
|---|---|---|---|
| 1. Confirm requirements | Review rubric, tool requirements, architecture, roles, and repository | 2–3 days | Team agrees on scope and implementation decisions |
| 2. Inventory service | Define data model and implement quantity tracking | 1 week | Core inventory operations pass tests |
| 3. Alert service and dashboard | Implement low-stock rule, alert behavior, and manager dashboard/reports | 1 week | Low-stock scenario and dashboard demonstrated |
| 4. Containerization | Create and test Docker images for services | 3–4 days | Services start from built images |
| 5. CI/CD | Configure Jenkins stages for checkout, build, tests, and deployment/update workflow | 4–5 days | Successful documented pipeline run |
| 6. Kubernetes | Configure and deploy services to Kubernetes | 1 week | Services are deployed and reachable as designed |
| 7. Infrastructure/configuration | Terraform for approved AWS resources; Ansible for server/service configuration | 1 week | Automation files execute successfully in approved environment |
| 8. Monitoring and access | Configure Prometheus metrics, Grafana views, and Nginx dashboard access | 4–5 days | Metrics and dashboard access demonstrated |
| 9. Integration and final review | End-to-end tests, security checks, documentation, and demo | 3–5 days | All required deliverables have reviewable evidence |

## 1.7 Proposed Team Responsibilities

The assignments below are proposed and should be confirmed by the four team members.

| Member | Workstream | Example tasks |
|---|---|---|
| Hazem El Sayed Abu El Wafa El Tayeb | System Architecture & Integration | Coordinate service boundaries and end-to-end integration |
| Mohamed Gamal Fawzy Abdelzaher | Inventory Tracking Service | Implement and test inventory workflows |
| Bishoy Emad Fawzy Zakhary | Docker & Jenkins CI/CD | Build container images and maintain the CI/CD pipeline |
| Mohamed Amer Farid Mohamed | Kubernetes & Infrastructure Automation | Manage Kubernetes deployment and infrastructure automation |
| All Team Members | Testing, Integration & Documentation | Test, integrate, and document the project |

## 1.8 Risks and Mitigation

| Risk | Impact | Mitigation |
|---|---|---|
| Required tools or rubric are | Rework or missing | Confirm the official rubric and clarify unclear details with |

| Risk | Impact | Mitigation |
|---|---|---|
| misunderstood | deliverables | the Technical Instructor |
| Scope is too large for the team/time | Incomplete system | Build the simplest demonstrable version of each required deliverable; defer unrequested extras |
| Cloud costs or access limitations | Deployment blocked or unexpected charges | Agree on AWS resources and budget first; use least-privilege credentials; destroy test resources when no longer needed |
| Secrets are committed to Git | Credential exposure | Never commit access keys, tokens, passwords, or private keys; use environment variables/secret management |
| Docker images or pipeline fail | Unreliable integration | Build and test images early; use small changes and keep pipeline logs |
| Kubernetes deployment is difficult to debug | Delayed demo | Validate each service locally before deploying; document manifests and troubleshooting steps |
| Monitoring targets are not reachable | Missing metrics | Configure and verify Prometheus targets early; record expected endpoints and ports |
| Team work becomes unbalanced | Bottlenecks | Assign owners and backups, track tasks, and conduct regular short reviews |
| Terraform/Ansible changes affect resources unexpectedly | Infrastructure disruption or cost | Review Terraform plan before apply, use a test environment, and inspect playbooks before running |

## 1.9 Proposed Success Metrics / KPIs

These are suggested acceptance targets; agree on final values with the team/instructor.

- **Inventory correctness:** all agreed create/read/update operations and quantity-validation tests pass.
- **Low-stock behavior:** all test cases flag products correctly according to the approved threshold rule.
- **Container reproducibility:** each required application service builds and starts from its documented Docker image.
- **CI/CD evidence:** at least three consecutive successful Jenkins runs on the agreed branch before the final demo (proposed target).
- **Kubernetes deployment:** all required workloads reach the intended healthy/running state in the approved test environment.
- **Monitoring:** Prometheus successfully collects the agreed system/container metrics; relevant data can be inspected in Grafana if included in the approved implementation.
- **Nginx access:** the manager dashboard is reachable through the configured Nginx entry point.
- **Infrastructure automation:** Terraform configuration and Ansible playbooks are reviewed and demonstrated in the approved environment.
- **Repository hygiene:** source and scripts are present in Git; no credentials are committed.
- **Documentation:** another team member can follow the documented build/deploy/monitoring steps.

## 1.10 Assumptions and Decisions to Confirm

1. The supplied Project 3 brief is the source of required deliverables for this revision.
2. Grafana appears in the supplied tool list; its exact required dashboard scope should be confirmed if the formal rubric specifies it.
3. Application framework, database, service API design, and service-to-service communication are not specified in the brief.
4. The precise Kubernetes environment and AWS topology are not specified.
5. The alert delivery channel is not specified.
6. The proposed schedule and individual roles are estimates, not official assignments.
7. Confirm how AWS resources should be handled and who owns the account/permissions before provisioning billable infrastructure.

# 2. Literature Review

## 2.1 Purpose and Method

This preliminary review compares established inventory-management systems to identify common inventory concepts relevant to MyShelf. It uses official product documentation as primary sources. It is a feature-oriented review, not a performance benchmark or proof that one system is universally better than another.

## 2.2 Comparable Systems

### A. ERPNext — Stock and Reorder Management

ERPNext documentation describes item-level reorder settings and stock-level reporting. Its stock concepts can include actual, planned, requested, ordered, reserved, and projected quantities [1][2].

**Strengths relevant to the comparison** - Supports reorder-level configuration and replenishment-oriented workflows. - Provides stock reports beyond a single current-quantity value. - Offers a useful reference for threshold-based stock management.

**Potential drawbacks for the MyShelf project scope** - ERPNext includes broader ERP and stock-management concepts that exceed the required minimum project features. - Reproducing purchasing and advanced replenishment workflows could increase implementation and testing effort.

**Lesson for MyShelf** - Make the low-stock threshold explicit and demonstrate the resulting alert. - Keep purchasing automation and advanced stock planning out of scope unless separately required.

### B. Odoo Inventory

Odoo's official Inventory documentation describes inventory and warehouse-management capabilities, including replenishment, lead-time considerations, product tracking, and warehouse/location concepts [3].

**Strengths relevant to the comparison** - Provides a broad reference for inventory and warehouse workflows. - Includes replenishment and product-tracking concepts. - Helps identify possible future extensions beyond the initial project.

**Potential drawbacks for the MyShelf project scope** - Advanced warehouse, route, and replenishment features may be more complex than necessary for a focused DevOps graduation project. - Replicating them would distract from the required automation and deployment work.

**Lesson for MyShelf** - Prioritize inventory tracking, low-stock alerts, and a manager dashboard. - Treat advanced warehouse operations and forecasting as future work unless the official rubric requires them.

## 2.3 Feature Comparison

| Capability | ERPNext (official docs) | Odoo Inventory (official docs) | MyShelf required/proposed scope |
|---|---|---|---|
| Item/product records | Supported | Supported | Inventory tracking service |
| Stock visibility/reporting | Stock-level reports | Inventory and warehouse views | Manager web dashboard and reports |
| Reorder/low-stock concepts | Reorder levels and stock reports | Replenishment workflows | Separate low-stock alert service |
| Advanced ERP/purchasing | Available within broader product scope | Broader business workflows available | Not required by supplied brief |
| Containerization and CI/CD | Not the focus of these product references | Not the focus of these product references | Explicit project deliverables |
| Kubernetes, Terraform, Ansible, monitoring | Not the focus of these product references | Not the focus of these product references | Explicit project deliverables |

This table compares documented feature areas, not every edition, extension, or deployment configuration.

## 2.4 Relevance to MyShelf

The supplied Project 3 brief defines MyShelf's intended emphasis as inventory tracking, low-stock alerts, reports, and DevOps automation. Unlike a full ERP product, MyShelf's objective is not to reproduce all business-management features. The application services provide a realistic workload for demonstrating Docker, Jenkins CI/CD, Kubernetes, Prometheus/Grafana monitoring, Nginx, Terraform, Ansible, AWS resources, and Git-based collaboration.

## 2.5 Literature Review Conclusion

The reviewed systems show that product records, stock visibility, threshold/replenishment concepts, and reporting are established inventory-management capabilities. MyShelf should implement a focused version of these capabilities while dedicating project effort to the explicitly required DevOps deliverables. Features such as advanced purchasing, multi-warehouse workflows, and AI forecasting are not required by the supplied brief and should not be added unless approved.

## 2.6 References

[1] Frappe Technologies, **ERPNext — Item (Automatic Reordering)**. https://docs.frappe.io/erpnext/item
[2] Frappe Technologies, **ERPNext — Stock Level Report**. https://docs.frappe.io/erpnext/stock-level-report
[3] Odoo S.A., **Odoo Inventory documentation**. https://www.odoo.com/documentation/20.0/applications/inventory_and_mrp/inventory.html
[4] Docker, **What is a container?** https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/
[5] Jenkins, **Pipeline documentation**. https://www.jenkins.io/doc/book/pipeline/
[6] Kubernetes, **Deployments**. https://kubernetes.io/docs/concepts/workloads/controllers/deployment/
[7] Prometheus, **Getting started**. https://prometheus.io/docs/tutorials/getting_started/
[8] Grafana, **Documentation**. https://grafana.com/docs/
[9] HashiCorp, **Terraform documentation**. https://developer.hashicorp.com/terraform/docs
[10] Ansible, **Getting started**. https://docs.ansible.com/projects/ansible/latest/getting_started/
[11] NGINX, **Beginner's Guide**. https://nginx.org/en/docs/beginners_guide.html
[12] Amazon Web Services, **Amazon EC2 documentation**. https://docs.aws.amazon.com/ec2/
[13] Amazon Web Services, **Amazon S3 documentation**. https://docs.aws.amazon.com/s3/
[14] Amazon Web Services, **Amazon VPC documentation**. https://docs.aws.amazon.com/vpc/

References [4]–[14] provide official technical descriptions of the tools and services named in the supplied project brief. They do not independently establish the institute's grading requirements; those requirements come from the supplied Project 3 brief.

---

# 3. Requirements Gathering

## 3.1 Stakeholders

| Stakeholder | Needs and interests |
|---|---|
| Inventory operator | Maintain product records and quantities; identify low-stock products |
| Manager | Review stock status, alerts, and inventory reports through the dashboard |
| DevOps/project team | Build, test, containerize, deploy, automate, and monitor services |
| System/cloud administrator | Configure approved AWS resources and server/service settings safely |
| Technical Instructor/evaluator | Verify the required deliverables, tool usage, documentation, and demonstration |

Authentication and role-specific permissions are not explicitly specified in the supplied brief and remain design decisions to confirm.

## 3.2 Functional Requirements — Application

Priority labels below indicate a proposed MVP ordering. The listed service types and dashboard are required by the supplied Project 3 brief; field-level details and acceptance rules are proposed where the brief is silent.

| ID | Requirement | Priority | Acceptance criterion |
|---|---|---|---|
| FR-01 | Inventory tracking service | Required | The service records and returns product inventory information |
| FR-02 | Track quantity changes | Required | An authorized/demo workflow can update quantity and the updated value is visible |
| FR-03 | Low-stock alert service | Required | The alert service identifies products meeting the approved low-stock condition |
| FR-04 | Manager web dashboard | Required | A manager can view inventory status and reports through a web interface |
| FR-05 | Generate inventory reports | Required by project description | The system provides at least the agreed basic stock report |
| FR-06 | Product data validation | Proposed | Required fields and quantity constraints are validated |
| FR-07 | Configure low-stock threshold | Proposed implementation detail | A threshold can be stored for each product or defined by an agreed rule |
| FR-08 | Search/filter products | Optional enhancement | Users can search by product name/code or filter low-stock items if time permits |
| FR-09 | Authentication and authorization | Confirm | Implement if required by the instructor/rubric or agreed system design |
| FR-10 | Email/SMS alert delivery | Confirm | Add only if explicitly required; the brief requires an alert service but does not specify its delivery channel |

## 3.3 Functional Requirements — DevOps and Infrastructure

These are directly mapped to the ten deliverables in the supplied Project 3 brief.

| ID | Requirement | Tool/technology named in brief | Acceptance evidence |
|---|---|---|---|
| DR-01 | Maintain code and scripts in a Git repository | Git | One shared repository contains source, configuration, scripts, and README |
| DR-02 | Containerize the inventory tracking service | Docker | Docker image builds and service starts successfully from the image |
| DR-03 | Containerize/deploy the required services as appropriate | Docker | Service container definitions and successful build/run evidence |
| DR-04 | Automate CI/CD updates | Jenkins | Pipeline configuration and successful automated update run |
| DR-05 | Deploy services on Kubernetes | Kubernetes | Deployment/service configuration and a working deployment demonstration |
| DR-06 | Monitor system and container metrics | Prometheus | Prometheus collects the agreed metrics and shows healthy targets |
| DR-07 | Visualize monitoring data | Grafana (listed in tools) | Proposed dashboard displays relevant metrics; confirm expected depth with instructor |
| DR-08 | Provide dashboard access | Nginx | Nginx configuration routes users to the web dashboard |
| DR-09 | Define/provision AWS infrastructure | Terraform; AWS EC2, S3, VPC | Terraform files and reviewed plan/apply evidence for approved resources |
| DR-10 | Automate server/service configuration | Ansible | Playbook(s) and successful configuration-run evidence |
| DR-11 | Document reproducible operations | Git README/runbook (proposed) | Another team member can follow documented setup, deployment, and monitoring steps |

## 3.4 Non-Functional Requirements

The source brief does not provide numeric quality targets. The following are proposed targets and should be agreed by the team and instructor.

| ID | Attribute | Proposed requirement |
|---|---|---|
| NFR-01 | Correctness | All agreed inventory and low-stock tests pass; quantity/threshold rules behave consistently |
| NFR-02 | Performance | For a demo dataset of up to 1,000 products, common list/update operations should normally complete within 2 seconds in the agreed test environment |
| NFR-03 | Reliability | Services can be restarted using documented procedures, and failures produce useful logs |
| NFR-04 | Scalability | Kubernetes configuration should allow the team to demonstrate the intended deployment/scaling capability required by the project |
| NFR-05 | Security | Do not commit secrets; restrict AWS IAM permissions; validate inputs; protect sensitive configuration |
| NFR-06 | Maintainability | Keep service code, manifests, pipeline, Terraform, and Ansible files organized and documented |
| NFR-07 | Observability | Prometheus collects the agreed system/container metrics; Grafana can present selected metrics if included in the approved implementation |
| NFR-08 | Reproducibility | Docker builds and deployment/configuration steps are documented and repeatable |
| NFR-09 | Usability | The dashboard clearly shows stock status, low-stock items, and agreed reports |
| NFR-10 | Cost awareness | AWS resources are provisioned only with team approval, monitored for cost, and removed when no longer needed |

## 3.5 Proposed Data Requirements

A proposed product record may include: - `product_id`: unique identifier - `sku`: product code (optional if not required) - `name`: product name - `category`: optional grouping - `quantity`: current available quantity - `minimum_stock_threshold`: threshold for low-stock detection - `updated_at`: timestamp of last update

The supplied brief does not prescribe the database, exact data model, or service API. These should be agreed before implementation.

## 3.6 Proposed Business Rules

1. Each product has a unique identifier.
2. Product name is required.
3. Quantity and threshold should be non-negative, unless the approved domain model states otherwise.
4. Proposed low-stock rule: `quantity <= minimum_stock_threshold`.
5. When inventory data changes, the alert service should evaluate the updated stock condition.
6. The dashboard should show the current inventory view and the agreed report(s).
7. The initial solution should use demo/synthetic data unless the team has permission to use real business data.
8. No AI is required; AI-based forecasting is not part of the proposed scope.

## 3.7 Proposed Service Boundaries

The project brief requires an inventory tracking service, an alert service, and a web dashboard. A simple proposed boundary is: - **Inventory Tracking Service:** owns product and quantity records and exposes the agreed operations. - **Low-Stock Alert Service:** evaluates stock against the approved threshold rule and creates alert status/events. - **Manager Dashboard:** presents inventory information, alert status, and reports. - **Nginx:** provides the dashboard access entry point. - **DevOps layer:** Jenkins pipeline, Docker images, Kubernetes deployment, Prometheus/Grafana monitoring, Terraform infrastructure, Ansible configuration, and Git repository.

The specific APIs, messaging method between services, and database arrangement are not specified by the source brief and must be selected by the team.

## 3.8 Constraints and Dependencies

- **Service dependency:** the alert service depends on reliable access to current inventory quantities and the approved threshold rule.
- **Dashboard dependency:** dashboard views and reports depend on the inventory service/API and alert-state interface being available.
- **Container/deployment dependency:** Docker images must build successfully before they can be deployed and updated through Kubernetes.
- **CI/CD dependency:** Jenkins requires repository access, build/test steps, and appropriately managed deployment credentials for any automated deployment stage.
- **Monitoring dependency:** Prometheus can collect only metrics exposed by reachable configured targets; required endpoints, ports, and scrape configuration must be defined and tested.
- **Infrastructure dependency:** Terraform execution requires an approved AWS account, permissions, region, and resource plan. EC2/S3/VPC provisioning must respect the team's budget and cleanup procedure.
- **Configuration dependency:** Ansible requires reachable target hosts and valid connection/privilege configuration.
- **Unspecified implementation choices:** the supplied brief does not prescribe a programming language, framework, database, service API, alert delivery channel, cluster type, or numeric performance target. These must remain proposed until agreed.
- **Security constraint:** credentials, access keys, tokens, passwords, and private keys must not be committed to Git. Use environment-specific secret handling and least-privilege access.
- **Team constraint:** implementation, testing, integration, and documentation are shared across a four-person team; service interfaces should be agreed early to reduce integration conflicts.

## 3.9 Technical Questions to Confirm with the Instructor

1. What acceptance evidence is required for each of the ten deliverables?
2. Is Grafana expected as a working visualization dashboard, given that it is in the tool list but not a separate numbered deliverable?
3. Which Kubernetes environment is approved (local, virtualized, or cloud-hosted), and what deployment/scaling demonstration is expected?
4. Must Terraform actually provision EC2, S3, and VPC resources in AWS, or is a validated plan/configuration sufficient? What account, region, and budget limits apply?
5. Is a dashboard/in-app low-stock alert sufficient, or must alerts be delivered through email/SMS or another external channel?
6. Are a specific language, framework, database, API style, or service communication pattern required?
7. Which application and infrastructure tests must be demonstrated, and is a minimum coverage target specified?
8. Are authentication, authorization, report export, and persistent audit/history data part of the assessed scope?

## 3.10 Technical Requirements Acceptance Checklist

For each requirement, record its status as **approved**, **in progress**, **verified**, or **not applicable with approval**. Numeric targets and implementation choices remain proposals unless explicitly stated in the supplied brief.

| Area | Verification evidence to collect |
|---|---|
| Inventory service | Demonstrate product creation/read/update and quantity validation using the agreed test cases |
| Low-stock alert | Demonstrate both above-threshold and low-stock scenarios using the agreed threshold rule |
| Dashboard and reports | Demonstrate current stock, low-stock status, and the agreed report(s) through the web interface |
| Docker | Build and run documented images; record successful startup and configuration requirements |
| Jenkins CI/CD | Show a pipeline run with checkout, validation/tests, image build, and the approved |

| Area | Verification evidence to collect |
|---|---|
| | update/deployment stages |
| Kubernetes | Show required workloads and service access in the approved environment; capture relevant status/output |
| Prometheus/Grafana | Show configured targets collecting metrics and, if required, a dashboard displaying selected metrics |
| Nginx | Demonstrate the dashboard through the configured Nginx access path |
| Terraform/AWS | Review Terraform configuration and plan; record approved apply evidence without exceeding budget/permissions |
| Ansible | Run the playbook against the approved target and verify the intended configuration change |
| Git/security | Verify code and scripts are versioned and that no credentials or private keys are committed |

## Final Technical Review Checklist

- ☐ Service boundaries, data flow, and API/database choices are documented and agreed.
- ☐ All ten Project 3 deliverables have an owner and demonstrable evidence.
- ☐ Build, test, deploy, monitoring, and infrastructure procedures are reproducible from the repository documentation.
- ☐ AWS permissions, budget, and resource cleanup are confirmed before provisioning.
- ☐ Repository history and configuration contain no secrets.
- ☐ Official technical references and the final PDF have been checked before submission.